

Advanced Topics in Artificial Intelligence

Exploring Federated Learning, Adaptive Feature Fusion, Octree Segmentation, DWT, and GAT

Group members

Name	ID
Abid Hasan	2023200000720
Joairiya Alam Sujana	2023200000719
Md. Fazley Rabby	2023200000686

Prepared for

Md. Mijanur Rahman

Assistant Professor

CSE 365.4

Artificial Intelligence

Date: 10 April 2026



Southeast University

Department of Computer Science and engineering

TABLE OF CONTENTS

Assignment Overview

This document contains five advanced topics in machine learning, signal processing, and computational geometry. Each section provides structured notes, mathematical foundations, implementation examples, and real-world applications.

No.	Assignment Topic	Page
01	Federated Learning	01 - 08
02	Adaptive Feature Fusion	09 - 13
03	Octree Segmentation	14 - 19
04	Discrete Wavelet Transform (DWT)	20 - 24
05	Graph Attention Networks (GAT)	25 - 29

Chapter 01: Federated Learning

A revolutionary paradigm that trains shared models across devices while keeping all raw data strictly local — privacy by design.

1.1 Introduction

Federated Learning (FL) is a revolutionary **decentralized machine learning** approach. It enables multiple devices — like smartphones, IoT devices, or hospital servers — to collaboratively train a shared predictive model while keeping all training data strictly local.

Instead of moving sensitive data to a centralized cloud database, FL brings the machine-learning algorithm directly to the device where the data resides. This paradigm shift ensures absolute data privacy, as raw data never leaves its origin.

1.2 Core Philosophy

The guiding principle of Federated Learning is “**Send the model to where the data resides, rather than moving the data to a central location for processing.**”

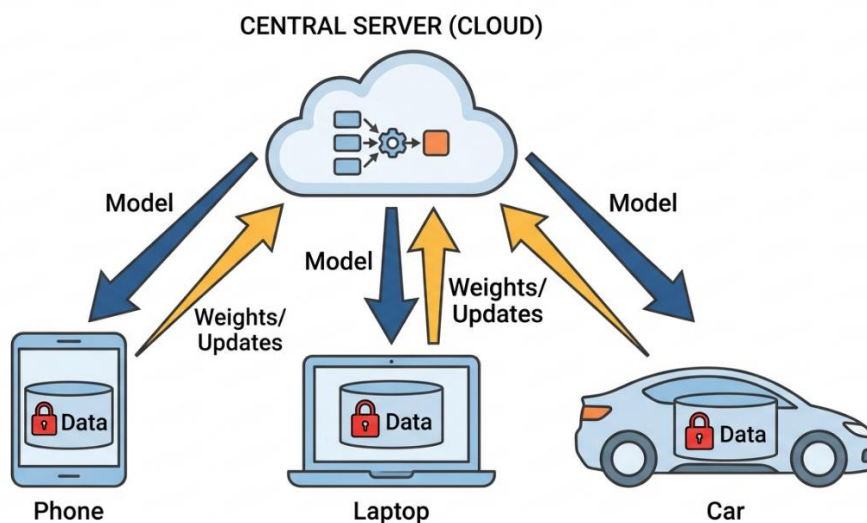


FIGURE 1_1 — FEDERATED LEARNING ARCHITECTURE: MODEL FLOWS
DOWN, ONLY MATHEMATICAL UPDATES FLOW UP

- **Traditional ML:** Uploads all user data (photos, texts, medical records) to a central server to train a model.

- **Federated ML:** Downloads the model to the user's device, learns locally from the user's behavior, and only uploads the mathematical "lessons learned" (**model weights/gradients**) back to the server.

1.3 Working Process (Step-by-Step)

The Federated Learning cycle operates in continuous rounds. Here is how a single round works:

- 1. Initialization:** The central server initializes a global model and broadcasts it to a subset of participating devices (clients).
- 2. Local Training:** Each client uses its own local dataset to train the downloaded model for a few epochs.
- 3. Local Update Transfer:** Instead of sharing the raw data, clients send only the updated model parameters (**weights/gradients**) securely back to the central server.
- 4. Server Aggregation:** The server aggregates these local updates (typically using an averaging algorithm) to construct a smarter, updated global model.
- 5. Global Update:** The server broadcasts this new global model back to the clients, and the cycle repeats until the model converges.

1.4 Mathematical Formulation: Federated Averaging (FedAvg)

The most standard aggregation algorithm in FL is **FedAvg**. Suppose we have K participating clients. Let n_k be the number of data samples on client k, and n be the total number of data samples across all participating clients.

$$w_{t+1} = \sum_{k=1}^K (n_k / n) \cdot w_{t+1}^k$$

Where: FedAvg Global Weight Formula

- $w_{(t+1)}$ → Updated global model (after aggregation on the server)
- K → Total number of clients/devices
- n_k → Number of data samples in the k-th client
- n → Total number of data samples across all clients

$$n = n_1 + n_2 + \dots + n_K$$
- $w_{k(t+1)}$ → Model weights trained by the k-th client using its local data

SIMPLE MATHEMATICAL SIMULATION

Given: Initial Global Weight (w_0) = 0.5

→ Client 1: $n_1 = 20$ data points, local weight $w^1 = 0.6$

→ Client 2: $n_2 = 30$ data points, local weight $w^2 = 0.8$

→ Total data $n = 50$

$$\begin{aligned}w_1 &= (20/50 \times 0.6) + (30/50 \times 0.8) \\&= (0.4 \times 0.6) + (0.6 \times 0.8) \\&= 0.24 + 0.48 = \mathbf{0.72}\end{aligned}$$

NEW GLOBAL WEIGHT AFTER AGGREGATION = 0.72

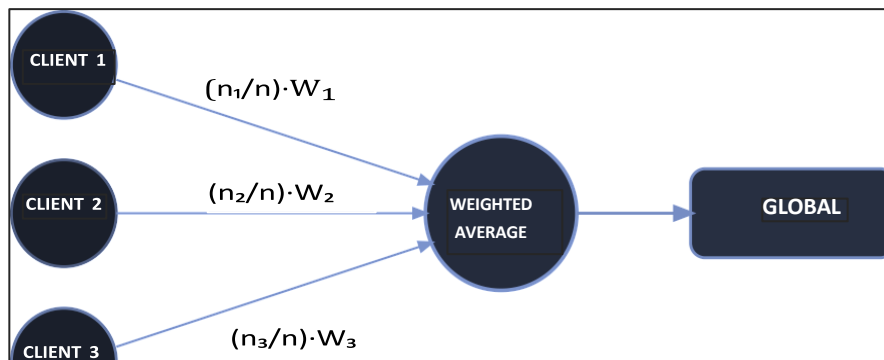


FIGURE 1.2 — FedAvg AGGREGATION: WEIGHTED CONTRIBUTIONS FROM ALL CLIENTS MERGE INTO ONE UPDATED GLOBAL MODEL

1.5 Algorithm: Pseudocode for FedAvg

PSEUDOCODE - FedAvg

Server executes:

Initialize global model weights W_0 For

each round $t = 1, 2, \dots$ do:

Select a random subset of clients S_t For

each client k in S_t (in parallel):

$W_k = \text{ClientUpdate}(k, W_t)$ *// Send global weights to client*

// Aggregate updates using weighted average

$W_{(t+1)} = \text{sum}((n_k / n) * W_k)$ for all k in S_t

ClientUpdate(k, W):

Set local model weights $w = W$

For each local epoch:

For each batch in local dataset:

$w = w - \text{learning_rate} * \text{gradient}(\text{loss}(w, \text{batch}))$

Return w to server

1.6 Python Implementation

```
import tensorflow as tf
import numpy as np

# 1. Build Model
def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Dense(1, input_shape=(2,), activation='linear')
    ])

    model.compile(
        optimizer='sgd',
        loss='mean_squared_error'
    )

    return model

# 2. Initialize Global Model
global_model = create_model()

# 3. Client Data (Local)
X_A = np.array([[1.0, 2.0], [2.0, 3.0]])
y_A = np.array([3.0, 5.0])

X_B = np.array([[3.0, 4.0], [5.0, 6.0]])
y_B = np.array([7.0, 11.0])

# 4. Train Client A
client_A = create_model()
```

```
client_A.set_weights(global_model.get_weights())
```

```
client_A.fit(X_A, y_A, epochs=5, verbose=0)
```

```
weights_A = client_A.get_weights()
```

```
# 5. Train Client B
```

```
client_B = create_model()
```

```
client_B.set_weights(global_model.get_weights())
```

```
client_B.fit(X_B, y_B, epochs=5, verbose=0)
```

```
weights_B = client_B.get_weights()
```

```
# 6. Federated Averaging (FedAvg)
```

```
new_weights = []
```

```
for wA, wB in zip(weights_A, weights_B):
```

```
    avg = (wA + wB) / 2
```

```
    new_weights.append(avg)
```

```
# 7. Update Global Model
```

```
global_model.set_weights(new_weights)
```

```
# 8. Display Results
```

```
print("Updated Global Weights:")
```

```
print(global_model.get_weights()[0])
```


1.7 Advantages & Challenges

Features	Details
Advantages	Privacy: Data never leaves the device. Efficiency: Sending weights uses far less bandwidth than uploading big data. Low Latency: Models can make real-time predictions locally.
Challenges	Communication Overhead: Sending models back and forth can drain battery. Non-IID Data: Data across users is imbalanced (one user types in English, another in Spanish), making global convergence tricky. Straggler Effect: Devices with slow internet or low battery delay the whole training round.

1.8 Real-World Applications



Healthcare

Hospitals collaborate to train tumor-detection models on MRI scans without violating HIPAA or sharing patient data.



Smart Keyboards

Gboard/Apple's next-word prediction trains on your personal typing history and only shares mathematical gradients.



Autonomous Vehicles

Self-driving cars learn from localized weather and road conditions and share the "driving logic" without uploading massive video files.

1.9 References for Further Reading

1. McMahan, B., et al. (2017). "**Communication-Efficient Learning of Deep Networks from Decentralized Data.**" Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS). (The foundational paper introducing FedAvg).
2. Kairouz, P., et al. (2021). "**Advances and Open Problems in Federated Learning.**" Foundations and Trends® in Machine Learning.
3. Li, T., et al. (2020). "**Federated Learning: Challenges, Methods, and Future Directions.**" IEEE Signal Processing Magazine.

Chapter 02: Adaptive Feature Fusion

An intelligent technique that dynamically weights and combines multiple information sources using learned attention mechanisms — not fixed rules.

2.1 Introduction

Adaptive Feature Fusion (AFF) is a dynamic and intelligent technique in deep learning where multiple information sources (features) are combined using weights that change based on the input data.

Unlike traditional methods that treat all incoming information as equally important, AFF intelligently selects or weights features to create a highly informative representation. This approach has become a cornerstone in advanced computer vision and multimodal learning (e.g., combining text, audio, and images).

2.2 The Core Philosophy

Core Principle — "**Dynamic Integration over Static Rules.**" The model learns how to blend information on the fly.

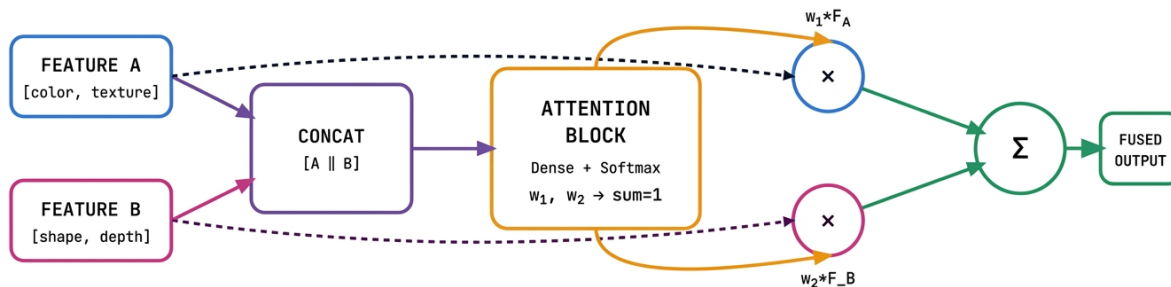


FIGURE 2.1 – AFF ARCHITECTURE: FEATURES ARE DYNAMICALLY WEIGHTED BY THE ATTENTION BLOCK BEFORE SUMMATION

2.3 Working Process (Step-by-Step)

The typical AFF process inside a neural network operates through these sequential steps:

- 1. Input Features Collection:** The module receives features from different sources. This could be multi-scale features (different layers of a CNN) or multimodal features (e.g., camera images + LiDAR sensor data).
- 2. Weighting Mechanism (Attention):** A small sub-network (such as a Multi-Layer Perceptron or an Attention block) analyzes these inputs to calculate dynamic gating coefficients or attention weights.
- 3. Dynamic Weighting:** The calculated weights are applied to the input features (usually via element-wise multiplication). This step emphasizes useful information while suppressing noise or redundancy.
- 4. Output Fusion:** The weighted features are combined (typically summed together) to produce a single, refined output feature map that is passed to the next layer.

2.4 Mathematical Formulation & Simulation

$$\begin{aligned}w_1, w_2 &= A(F_1, F_2) \\ F_{\text{fused}} &= (w_1 \cdot F_1) + (w_2 \cdot F_2) \\ \text{ADAPTIVE FUSION WITH CONTEXT-DEPENDENT WEIGHTS}\end{aligned}$$

CONCEPTUAL SIMULATION: NIGHT-TIME OBJECT DETECTION

Scenario: F_A = Color features [0.3, 0.5], F_B = Shape features [0.2, 0.8]

At night, shape matters more. Network assigns: $w_1 = 0.30$ (color), $w_2 = 0.70$ (shape)

$$\begin{aligned}w_1 * F_A &= 0.30 * [0.3, 0.5] = [0.09, 0.15] \\ w_2 * F_B &= 0.70 * [0.2, 0.8] = [0.14, 0.56] \\ F_{\text{fused}} &= [0.09+0.14, 0.15+0.56] = [0.44, 0.71] \\ &\text{Shape information (} w_2=0.7 \text{) dominates — correct for dim lighting!}\end{aligned}$$

2.5 Algorithm: Pseudocode for AFF Block

Pseudocode · AFF Block

```
Function AdaptiveFeatureFusion(Feature_A, Feature_B):  
    // Step 1: Concatenate features to understand the combined context  
    Combined_Features = Concatenate(Feature_A, Feature_B)  
  
    // Step 2: Pass through a small network to generate raw scores  
    Raw_Scores = DenseNetwork(Combined_Features)  
  
    // Step 3: Apply Softmax to get normalized weights (sum to 1)  
    Weights_A, Weights_B = Softmax(Raw_Scores)  
  
    // Step 4: Multiply original features by their respective weights  
    Weighted_A = Feature_A * Weights_A  
    Weighted_B = Feature_B * Weights_B  
  
    // Step 5: Sum them up for the final fused output  
    Fused_Output = Weighted_A + Weighted_B  
  
    Return Fused_Output
```

2.6 Python Implementation

```
import tensorflow as tf  
from tensorflow import keras  
import numpy as np  
  
# Input features  
f1 = keras.Input(shape=(2,))  
f2 = keras.Input(shape=(2,))  
  
# Combine features  
combined = keras.layers.Concatenate()([f1, f2])  
  
# Generate attention weights (2 values)  
weights = keras.layers.Dense(2, activation='softmax')(combined)  
  
# Split weights  
w1 = weights[:, 0:1]  
w2 = weights[:, 1:2]  
  
# Apply weights
```

```

f1_weighted = keras.layers.Multiply()([f1, w1])
f2_weighted = keras.layers.Multiply()([f2, w2])

# Final fusion
output = keras.layers.Add()([f1_weighted, f2_weighted])

# Model
model = keras.Model(inputs=[f1, f2], outputs=output)

# Dummy data
f1_data = np.array([[1.0, 0.5]])
f2_data = np.array([[0.2, 0.8]])

# Prediction
result = model.predict([f1_data, f2_data], verbose=0)

print("Feature 1:", f1_data[0])
print("Feature 2:", f2_data[0])
print("Fused Output:", result[0])

```

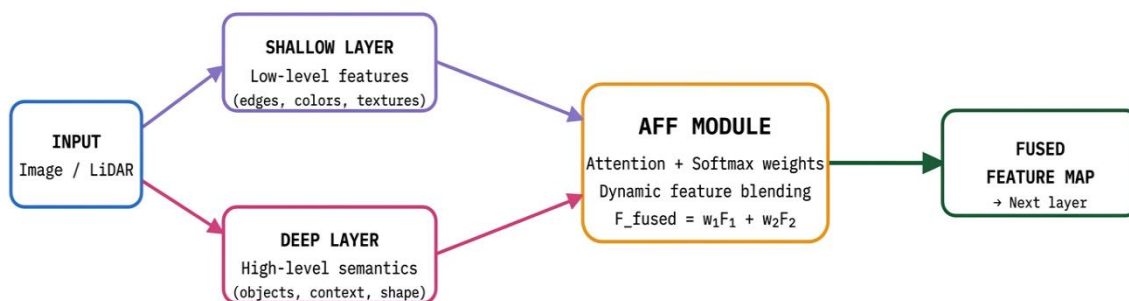


FIGURE 2.2 – MULTI-BRANCH NEURAL NETWORK WITH AFF: SHALLOW AND DEEP FEATURES DYNAMICALLY MERGED

2.7 Advantages & Limitations

Aspect	Details
Enhanced Generalization	Adapts dynamically to unseen data characteristics, generalizing better than fixed fusion.
Robustness to Noise	Automatically down-weights noisy or unreliable sensor inputs.
Feature Complementarity	Perfectly blends multi-scale (local vs global) and multimodal (text vs image) data.
Computational Overhead	The extra attention/gating networks increase parameters and slightly slow down inference.
Tuning Sensitivity	If poorly trained, the attention module might "collapse" and constantly assign a weight of 1.0 to only one feature branch.

2.8 Real-World Applications

- **Medical Imaging:** In skin lesion or tumor segmentation, AFF dynamically shifts focus between "local details" (blurry boundaries) and "global context" (overall tumor shape) depending on the clarity of the MRI/X-ray.
- **Autonomous Driving:** Self-driving cars use AFF to fuse LiDAR (depth) and Camera (RGB) data. If driving into blinding sunlight, the model dynamically lowers the camera's weight and increases the LiDAR's weight.
- **Multimodal AI (VQA):** In Visual Question Answering, the system dynamically fuses image features and text features based on what the question is asking.

2.9 References for Further Reading

01. Dai, Y., et al. (2021). "**Attentional Feature Fusion.**" IEEE WACV. (Foundational paper on robust AFF modules)
02. Li, X., et al. (2019). "**Selective Kernel Networks.**" IEEE CVPR. (Dynamic selection of receptive fields)
03. Hu, J., et al. (2018). "**Squeeze-and-Excitation Networks.**" (Pioneer paper for channel-wise attention)

Chapter 03: Octree Segmentation

A hierarchical spatial data structure that recursively divides 3D space into eight octants, enabling efficient processing of massive point clouds with $O(\log n)$ queries.

3.1 Introduction

Octree Segmentation is a highly efficient spatial partitioning technique used to organize and manage 3D data. It works by dividing a large 3D volume into smaller sections using a hierarchical tree structure where every internal node branches out into exactly eight children (octants).

3.2 The Core Philosophy

Core Principle — "**Recursive Spatial Subdivision.**" Empty spaces are left as large, undivided blocks saving massive amounts of memory and computation.

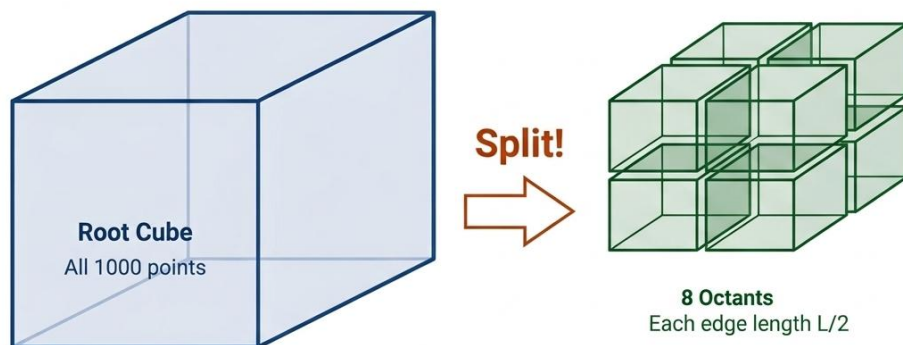


FIGURE 3.1 - OCTREE SUBDIVISION: ONE LARGE ROOT CUBE SPLITS INTO EXACTLY 8 EQUAL CHILD CUBES (OCTANTS)

3.3 Working Process (Step-by-Step)

The construction of an Octree follows a recursive top-down approach:

1. The Root Node: The process begins with a single large bounding box (the "root" cube)

that entirely encloses all 3D data points.

2. The Split Condition: The algorithm evaluates a condition—for example, "Does this cube contain more than N points?" or "Have we reached the maximum allowed depth?"

3. Subdivision (Splitting): If the condition is met, the cube is evenly divided along its X, Y, and Z axes, resulting in 8 smaller child cubes (octants).

4. Recursion: The algorithm sorts the data points into these new octants and repeats the splitting process for each non-empty octant.

5. Termination: The recursion stops when an octant meets the stopping criteria (e.g., becomes empty, contains very few points, or reaches maximum depth). These final undivided cubes are called "Leaf Nodes" or "Voxels".

3.4 Mathematical Formulation

Let a 3D bounding box be defined by center $C = (x_c, y_c, z_c)$ and side length L . When split:

$$L_{child} = L / 2$$
$$C_{child} = (x_c \pm L/4, y_c \pm L/4, z_c \pm L/4)$$

8 new child centers generated by $\pm$ combinations on all 3 axes

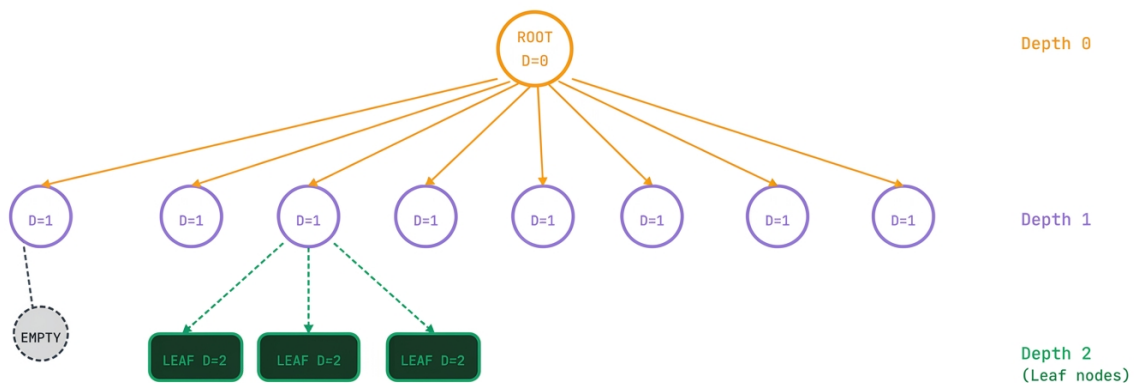


FIGURE 3.2 - OCTREE HIERARCHY: ROOT BRANCHES INTO 8, THEN 8 AGAIN RECURSIVELY. EMPTY OCTANTS ARE NOT EXPANDED (SAVES MEMORY)

3.5 Pseudocode for Octree Construction

Pseudocode · Octree Build

```
Function BuildOctree(Cube, Points, Depth, Max_Depth, Max_Points):  
    // Base Case: Stop if max depth reached or few points  
    If Depth == Max_Depth OR Points.Count <= Max_Points:  
        Return LeafNode(Cube, Points)  
  
    SubCubes = DivideInto8(Cube)  
    Children = []  
  
    For each sub_cube in SubCubes:  
        PointsInSubCube = GetPointsInside(Points, sub_cube)  
  
        // Only branch if sub-cube has points (saves memory!)  
        If PointsInSubCube.Count > 0:  
            ChildNode = BuildOctree(sub_cube, PointsInSubCube, Depth + 1, Max_Depth,  
Max_Points)  
            Children.Append(ChildNode)  
  
    Return InternalNode(Cube, Children)
```

3.6 Python Implementation

```
import numpy as np  
  
# Node  
class OctreeNode:  
    def __init__(self, center, size, points, depth=0):  
        self.center = center  
        self.size = size  
        self.points = points  
        self.depth = depth  
        self.children = []
```

Build tree

```
def build(node, max_depth, max_points):

    if node.depth >= max_depth or len(node.points) <= max_points:
        return node

    half = node.size / 2
    quarter = node.size / 4

    offsets = [
        (1,1,1), (1,1,-1), (1,-1,1), (1,-1,-1),
        (-1,1,1), (-1,1,-1), (-1,-1,1), (-1,-1,-1)
    ]

    for dx, dy, dz in offsets:

        new_center = node.center + np.array([dx, dy, dz]) * quarter

        cond_x = (node.points[:,0] >= node.center[0]) if dx > 0 else (node.points[:,0] < node.center[0])
        cond_y = (node.points[:,1] >= node.center[1]) if dy > 0 else (node.points[:,1] < node.center[1])
        cond_z = (node.points[:,2] >= node.center[2]) if dz > 0 else (node.points[:,2] < node.center[2])

        child_pts = node.points[cond_x & cond_y & cond_z]

        if len(child_pts) > 0:
            child = OctreeNode(new_center, half, child_pts, node.depth + 1)
            node.children.append(build(child, max_depth, max_points))

    return node
```

Print tree (IMPORTANT for understanding)

```
def print_tree(node):
    print("Depth:", node.depth,
          "| Points:", len(node.points),
          "| Children:", len(node.children))

    for child in node.children:
        print_tree(child)
```

```

# ----- Simulation -----

np.random.seed(42)
points = np.random.uniform(0, 100, (1000, 3))

center = np.array([50.0, 50.0, 50.0])
size = 100.0

root = OctreeNode(center, size, points)

print("Building Octree...")
tree = build(root, max_depth=3, max_points=50)
print("Done!\n")

# Show structure
print_tree(tree)

```

3.7 Advantages & Limitations

Aspect	Details
Memory Efficiency	Ignores empty spaces; memory is only allocated where data exists.
Fast Querying	Searching for a specific point is $O(\log n)$ because you can skip massive empty areas.
Scalability	Handles gigantic datasets like city-wide LiDAR scans easily.
Blocky Edges	At lower resolutions, surface boundaries look like zig-zags (voxelization effect).
Implementation Complexity	Building the tree efficiently requires complex pointer management and careful memory allocation.
Dynamic Data Issue	If objects move (like in an animation), the tree often needs to be entirely rebuilt.

3.8 Real-World Applications

- **3D Scanning & LiDAR:** Used in autonomous vehicles and drones to quickly map and segment environments (roads, trees, buildings) from laser scans.
- **Computer Graphics & Video Games:** Used for "Frustum Culling" and fast collision detection (e.g., quickly calculating if a bullet hit a wall without checking every single polygon).
- **Medical Imaging:** Efficiently rendering and storing high-resolution 3D MRI or CT scans, allowing doctors to zoom into specific dense tissues.

3.9 References for Further Reading

- 01.** Meagher, D. (1982). "**Geometric modeling using octree encoding.**" Computer Graphics and Image Processing. (The foundational paper)
- 02.** Samet, H. (1990). "**The Design and Analysis of Spatial Data Structures.**" Addison-Wesley.
- 03.** Elseberg, J., et al. (2013). "**One billion points in the cloud.**" ISPRS Journal of Photogrammetry and Remote Sensing.

Chapter 04: Discrete Wavelet Transform

A powerful mathematical tool for time-frequency signal analysis that captures both when and what frequency anomalies occur the foundation of JPEG 2000 compression.

4.1 Introduction

The Discrete Wavelet Transform (DWT) is a powerful mathematical tool used to analyze digital signals. It quantifies how much energy is contained in specific frequency bands at particular moments in time. Unlike the Fourier Transform, DWT provides both time and frequency information simultaneously.

4.2 The Core Philosophy

Core Principle — Time-Frequency Localization. FFT tells you what frequencies exist; DWT tells you when they occur.

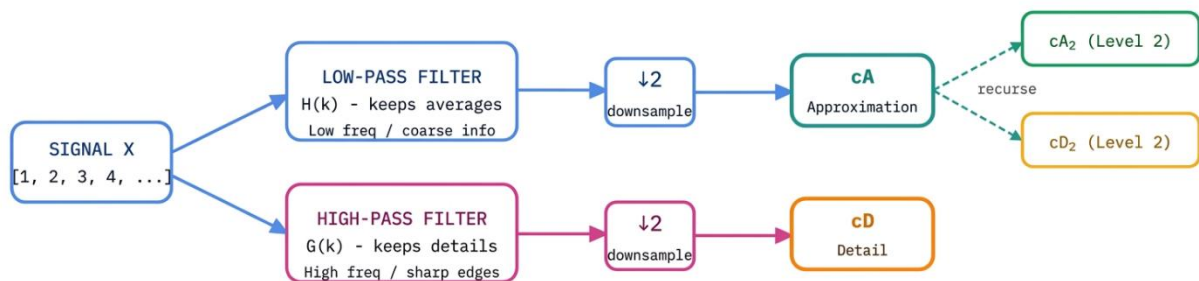


FIGURE 4.1 – DWT FILTER BANK (PYRAMID ALGORITHM): SIGNAL SPLITS INTO APPROXIMATION (CA) AND DETAIL (CD) AT EACH LEVEL, THEN CA IS RECURSIVELY PROCESSED

4.3 Working Process: The Pyramid Algorithm

1. **INPUT** — The raw 1D digital data is fed into the system.
2. **LOW-PASS FILTER (cA)** — Extracts “coarse” information — the general trend or average of the data.
3. **HIGH-PASS FILTER (cD)** — Captures “fine” details — sudden spikes or sharp edges.
4. **DOWNSAMPLING (↓2)** — Since the frequency band has halved, every second sample is safely discarded (Nyquist rule).

5.MULTI-LEVEL DECOMPOSITION — Steps 1-4 repeat recursively only on the Approximation output, creating a cascading tree.

4.4 Mathematical Formulation (Haar Wavelet)

$$cA_i = (x_{2i-1} + x_{2i}) / \sqrt{2}$$

$$cD_i = (x_{2i-1} - x_{2i}) / \sqrt{2}$$

Haar Wavelet — simplest wavelet basis. sqrt(2) scaling conserves signal energy.

STEP-BY-STEP HAAR DWT ON X = [1, 2, 3, 4]

Process pairs (1,2) and (3,4) separately:

```
Approximation (cA):  
(1+2)/√2 ≈ 2.121 | (3+4)/√2 ≈ 4.950 cA = [2.121, 4.950]  
Detail (cD):  
(1-2)/√2 ≈ -0.707 | (3-4)/√2 ≈ -0.707  
cD = [-0.707, -0.707]
```

4.5 Algorithm: Pseudocode for 1D Haar DWT

Here is how the algorithm works logically without complex libraries:

Plaintext

```
Function Haar_DWT_1D(Signal):
```

```
Length = size(Signal)
```

```
cA = empty_list() // Approximation
```

```
cD = empty_list() // Detail
```

```
// Process pairs of numbers (stride of 2)
```

```
For i from 0 to Length-1 step 2:
```

```
val1 = Signal[i]
```

```
val2 = Signal[i+1]
```

```
// Calculate and apply energy scaling factor
```

```
approx = (val1 + val2) / sqrt(2)
```

```
detail = (val1 - val2) / sqrt(2)
```

```
cA.append(approx)
```

```
cD.append(detail)
```

```
Return cA, cD
```

4.6 Python Implementation (PyWavelets)

```
#Python · PyWavelets

import numpy as np
import pywt

# 1. Define the input signal
signal = np.array([1, 2, 3, 4])
print("Original Signal:", signal)

# 2. Apply 1D DWT using Haar wavelet
# cA = Approximation Coefficients (Low frequency)
# cD = Detail Coefficients (High frequency)
cA, cD = pywt.dwt(signal, 'haar')

print("\n--- Level 1 DWT Coefficients ---")
print(f"Approximation (cA): {np.round(cA, 3)}") # [2.121, 4.950]
print(f"Detail (cD): {np.round(cD, 3)}") # [-0.707, -0.707]

# 3. Reconstruction (Inverse DWT) – proves perfect reconstruction
reconstructed = pywt.idwt(cA, cD, 'haar')
print(f"\nReconstructed Signal: {np.round(reconstructed, 1)}") # [1, 2, 3, 4]
```


4.7 Advantages & Limitations

Aspect	Details
Perfect Localization	Tells us exactly where a sudden frequency change (glitch or spike) occurs in time.
Sparsity	In natural images/audio, most detail coefficients become zero — perfect for high-ratio data compression.
Computational Speed	Filter bank implementation takes only $O(N)$ time — faster than FFT's $O(N \log N)$.
Shift Sensitivity	Standard DWT is sensitive to time alignment; a tiny 1-sample shift can drastically change resulting coefficients.
Power of Two Requirement	Basic algorithms require signal length to be exactly 2^n ; otherwise, padding with zeros is required.

4.8 Real-World Applications

- **Data Compression (JPEG 2000):** DWT is the core engine behind the JPEG 2000 image standard, offering better compression without the "blocky" artifacts seen in standard JPEG.
- **Signal De-noising:** Used to remove background hiss from audio or grain from photos by simply forcing very small Detail Coefficients, cD to zero before reconstructing the signal.
- **Biomedical Engineering:** Categorizing and analyzing sharp anomalies in ECG (heart monitoring) or EEG (brainwave) data.
- **Telecommunications:** Used in OFDM systems and wireless channel coding to handle interference robustly.

4.9 REFERENCES FOR FURTHER READING

- 01.** Mallat, S. G. (1989). "**A Theory for Multiresolution Signal Decomposition.**" IEEE TPAMI. (Definitive Filter Bank paper)
- 02.** Daubechies, I. (1992). "**Ten Lectures on Wavelets.**" SIAM. (The ultimate wavelet mathematics textbook)
- 03.** Lee, T. S. (1996). "**Image Representation Using 2D Gabor Wavelets.**" IEEE TPAMI.

Chapter 05: Graph Attention Networks

A powerful graph neural network architecture that dynamically assigns importance weights to each neighbor node — enabling interpretable, attention-driven representation learning on graphs.

5.1 Introduction

Graph Attention Networks (GAT) represent a highly powerful and advanced class of neural architectures within the broader field of Graph Neural Networks (GNNs). They are designed specifically to operate on graph-structured data, such as social networks, molecular structures, and citation networks. The defining feature of a GAT is its use of an "Attention Mechanism," which allows a node to dynamically decide which of its neighbors' information is the most critical and relevant for its own representation.

5.2 The Core Philosophy

Core Principle — "Dynamic Attention over Static Aggregation."

- **Standard GNNs (e.g., GCNs):** Typically aggregate information from all neighboring nodes with equal importance or based on a fixed structural rule.
- **GATs:** Introduce the ability to assign different importance (weights) to different neighbors. This means a node can focus heavily on neighbors that hold highly relevant features while effectively ignoring noise from less important ones.

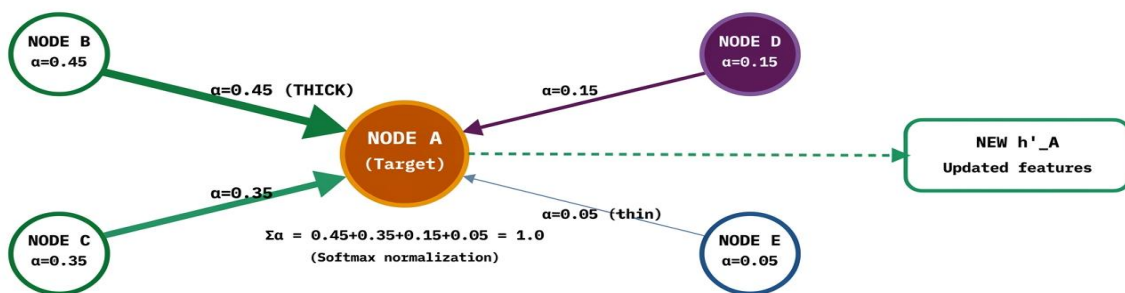


FIGURE 5.1 - GAT ATTENTION MECHANISM: ARROW THICKNESS = ATTENTION WEIGHT. NODES B AND C ARE MOST IMPORTANT; E IS MOSTLY IGNORED ($\alpha=0.05$)

5.3 Working Process (Step-by-Step)

1. **Linear Transformation:** To increase the expressive power of the model, each node's initial features are multiplied by a shared learnable weight matrix (W). This projects the data into a higher-level feature space.
2. **Attention Coefficient Calculation:** The model calculates a raw attention score (e_{ij}) between a target node i and its neighbor j . This is achieved by concatenating their transformed features and passing them through a shared linear layer followed by a LeakyReLU activation.
3. **Normalization (Softmax):** To make the scores comparable across different nodes, the raw scores are passed through a Softmax function. This ensures that the final attention weights (α_{ij}) sum up to exactly 1.
4. **Weighted Aggregation:** Finally, the node calculates its new, updated feature representation (h'_i) by multiplying the transformed features of its neighbors by their respective attention weights and summing them up.

5.4 Simple Conceptual Simulation

3-NODE GRAPH: NODE A UPDATES FROM B AND C

Attention mechanism decides: Node B ($\alpha=0.8$) is highly relevant, Node C ($\alpha=0.2$) less so.

$$\begin{aligned} z_B &= [1.0, 2.0] & | & \alpha_{AB} = 0.8 \\ z_C &= [3.0, 4.0] & | & \alpha_{AC} = 0.2 \\ 0.8 \times [1.0, 2.0] &= [0.8, 1.6] \\ 0.2 \times [3.0, 4.0] &= [0.6, 0.8] \\ h'_A &= [0.8+0.6, 1.6+0.8] = [1.4, 2.4] \end{aligned}$$

5.5 Pseudocode for GAT Layer

Below is the logical pseudocode demonstrating how a single Graph Attention layer processes node features:

```
Plaintext
Function GAT_Layer(Node_Features, Graph_Edges):
    W = Learnable_Weight_Matrix
    a = Attention_Mechanism_Parameters

    // 1. Apply linear transformation to all node features
    z = Node_Features * W

    For each Node i in Graph:
        Raw_Scores = empty_list()
```

```

For each Neighbor j of Node i:
    // 2. Calculate raw attention score between Node i and Neighbor j
    e_ij = LeakyReLU(a_transpose * Concatenate(z_i, z_j))
    Raw_Scores.append(e_ij)

    // 3. Normalize the raw scores to get attention weights (summing to 1)
    Alpha_i = Softmax(Raw_Scores)

    // 4. Calculate the new feature representation using weighted aggregation
    h_prime_i = ELU(Sum_over_all_j(Alpha_ij * z_j))

Return h_prime (The updated features for all nodes)

```

5.6 Python Implementation

```

import tensorflow as tf
import numpy as np

# Node features (A, B, C)
h = tf.constant([
    [1.0, 1.0], # A
    [1.0, 2.0], # B
    [3.0, 4.0]  # C
])

# Linear transformation (W * h)
W = tf.keras.layers.Dense(2, use_bias=False)
z = W(h)

# Raw attention scores (A -> B, C)
scores = tf.constant([1.5, 0.1])

# Softmax (normalize)
alpha = tf.nn.softmax(scores)
print("Attention Weights:", alpha.numpy())

# Take neighbor features (B, C)

```

```

neighbors = z[1:]

# Weighted aggregation
alpha = tf.reshape(alpha, (-1, 1))
new_A = tf.reduce_sum(alpha * neighbors, axis=0)

# Activation (ELU)
final_A = tf.nn.elu(new_A)

print("New Representation for Node A:", final_A.numpy())

```

5.7 Advantages & Limitations

Aspect	Details
Parallelization	Operations are highly parallelizable across all nodes and edges — computationally efficient.
Interpretability	Researchers can inspect attention weights (α) to understand exactly which neighbors drove predictions.
Inductive Learning	GAT can be applied to entirely new, unseen graphs not present during training.
Static Attention	Original GAT can suffer from "static attention" — neighbor ranking stays the same regardless of query context. Addressed in GATv2.
Oversmoothing	Reaching distant nodes requires stacking many layers, which can make all node representations look identical.

5.8 Real-World Applications

Citation Networks — Predicting the topic of a scientific paper based on the network of papers it cites (Cora, Citeseer datasets).

Bioinformatics — Analyzing Protein-Protein Interaction networks to predict cellular functions or identify drug targets.

Social Networks — Detecting communities, predicting user behavior, or filtering malicious bot rings in massive social graphs.

Traffic Forecasting — Modeling road intersections as graph nodes to predict future traffic jams based on local neighborhood flow.

5.9 REFERENCES FOR FURTHER READING

01. Velickovic, P., et al. (2018). "**Graph Attention Networks.**" ICLR. (The foundational GAT paper)
02. Brody, S., et al. (2022). "**How Attentive are Graph Attention Networks?**" ICLR. (Proposes GATv2 — fixes static attention limitation)
03. Hamilton, W. L. (2020). "**Graph Representation Learning.**" Morgan & Claypool Publishers.

Code Implementation

All practical implementations for this assignment are available in a single Google Colab notebook. [Open Full Notebook](#)

— End of Document —

Advanced Machine Learning Study Material • 5 Topics • FL • AFF • Octree • DWT • GAT